

TP n°1 : Expressions régulières

Objectif : utiliser un outil basé sur les expressions régulières.

Partie 1 : Python regular expressions

Se familiariser avec la bibliothèque *Python regular expressions*, `regex`, en lisant notamment les pages de tutoriel proposées sur l'ENT (dans Moodle) et en faisant des tests. On observera au passage que la notation utilisée ici pour les expressions régulières est différente de la notation formelle utilisée en cours et en TD. Cependant, cette notation est partagée par de nombreux outils, et elle vaut donc la peine d'être retenue. Identifier les étapes principales de l'analyse d'un texte par une expression régulière (quelles classes et méthodes Python ?).

Écrire un programme Python effectuant un test élémentaire (sans lecture de fichier, ni lecture au clavier) : analyse d'une chaîne de caractère avec une expression régulière simple. Tester les expressions régulières du TD, en particulier les exemples et contre-exemples.

Partie 2 : Recherche d'instances d'une expression régulière

Écrire des expressions régulières et en chercher les instances dans un texte. Par exemple, la séquence `a23 getrdHellofest>Helloss` contient 2 occurrences du mot `Hello`.

Écrire un programme Python qui lit le fichier `testRegEx_TP1.txt` (disponible sur Moodle) — le fichier sera traité ligne par ligne — et qui

- calcule le nombre d'occurrences du mot `hello` dans le fichier (NB : le mot peut commencer par une majuscule) ;
- construit une liste de tous les numéros de téléphone (du format `xx xx xx xx xx` où `x` est un chiffre) présents dans le fichier ;
- construit la liste de tous les numéros de téléphone qui commencent par `11`.

Partie 3 : Validation de mots de passe avec *look-ahead*

Écrire des expressions régulières pour contrôler les mots de passe. La ligne en entrée devra **correspondre entièrement** à l'expression régulière pour être validée.

1. Expression régulière d'un mot de passe simple : accepte tout mot d'au moins 6 caractères construit sur l'alphabet : minuscules, majuscules, chiffres plus 4 caractères spéciaux de votre choix.

2. Expression régulière d'un mot de passe élaboré : accepte tout mot du type précédent, vérifiant de plus la contrainte suivante. Le mot contient au moins un des caractères spéciaux, au moins une lettre et au moins un chiffre. On utilisera pour cela des assertions de lecture en avant, *look-ahead*, contrôlant l'expression régulière.

Votre programme commencera par un test élémentaire, puis par un test en série par lecture des différentes propositions de mots de passe inscrites dans un fichier. Le programme affichera chaque proposition et le résultat de son analyse.

3. Reprendre des exemples du TD qui auraient bénéficié de la possibilité de noter une soustraction ensembliste et les coder en Python plus *look-ahead*. Qu'en conclure ?